计算机学院

数据安全实验报告

实验 1: 半同态加密应用实践

姓名：王旭

学号：2312166

专业：计算机科学与技术

2026 年 3 月 31 日

目录

1 实验名称 **2**

2 实验目的 **2**

3 实验原理 **2**

3.1 隐私信息获取 (PIR) 与 Paillier 加密基本概念 2

3.2 Paillier 的同态特性 2

3.3 隐私信息获取 (PIR) 核心机制 3

3.4 扩展实验中的对称加密结构与数据类型转换 3

4 实验环境配置流程 **3**

4.1 安装与配置 phe 库 3

4.2 安装与配置 cryptography 库 4

5 基础实验 **4**

5.1 实现流程与思路 4

5.2 具体实现代码 5

5.3 代码运行结果与分析 6

6 扩展实验 **7**

6.1 实现流程与思路 7

6.2 具体实现代码 8

6.3 代码运行结果与分析 10

7 实验总结 **11**

7.1 主要工作 11

7.2 实验心得 12

8 github 仓库 **12**

1 实验名称

半同态加密应用实践

2 实验目的

- **基础实验**：基于 Paillier 算法实现隐私信息获取：从服务器给定的 m 个消息中获取其中一个，不得向服务器泄露获取了哪一个消息，同时客户端能完成获取消息的解密。
- **扩展实验**：在客户端保存对称密钥 k，在服务器端存储 m 个用对称密钥 k 加密的密文，通过隐私信息获取方法得到指定密文后能解密得到对应的明文。

3 实验原理

3.1 隐私信息获取 (PIR) 与 Paillier 加密基本概念

在常规的网络请求中，客户端要获取服务器数组 `[A, B, C]` 中的 `B`，只会简单发送一个请求（如 `GET index=1`）。服务器收到后，把 `B` 返回给客户端。但是，在这个过程中，服务器清楚地知道客户端拿的是第二个元素。

隐私信息获取（PIR）的关键是：客户端拿到了指定的元素，而服务器虽然参与了整个提取过程，但是并不知道客户端实际获取的是哪一个位置的数据。

为了实现这一目标，本次实验引入了 Paillier 加密算法。它是一种非对称加密（公钥加密）体制，包含一对核心的密钥：

- **公钥 (Public Key)**：公开用于加密数据。Paillier 在加密中会引入随机数，使针对同一个明文每次生成的密文完全不同（具有语义安全性），从而阻止服务器通过比对密文来推测明文的内容。
- **私钥 (Private Key)**：由客户端私密保存，用于将接收到的密文解密还原为原始明文。

3.2 Paillier 的同态特性

Paillier 算法最核心的特性在于，它具备“加法同态”以及衍生的“标量乘法”特性。这使得服务器可以在不了解明文内容的情况下，直接对密文进行算术运算：

- **特性 A（密文相加 = 明文相加）**：把明文 x 加密成 $E(x)$，明文 y 加密成 $E(y)$。如果在密文空间里对这两个密文进行同态加法运算，解密后得到的结果等于 $x+y$。即：$E(x) \oplus E(y) \to E(x+y)$。
- **特性 B（密文乘常量 = 明文乘常量）**：把明文 x 的密文 $E(x)$ 乘以一个未加密的普通数值 k，解密后得到的是原明文与该常数相乘的结果。即：$E(x) \otimes k \to E(x \times k)$。即 **“标量乘的性质”**。

结合“特性 B”，我们可以推导出以下关键逻辑：

- 数值“0”的密文与任意普通数字 M 的标量乘依然是 0。即：$E(0) \otimes M \to E(0 \times M) = E(0)$。（任何数乘 0 依然是 0）
- 数值“1”的密文与任意普通数字 M 的标量乘将是该数值本身。即：$E(1) \otimes M \to E(1 \times M) = E(M)$。（任何数乘 1 还是本身）

利用这一点，客户端可以将自己的“选择向量”加密后交给服务器。

3.3 隐私信息获取 (PIR) 核心机制

利用上述“0 和 1”的标量乘特性，当客户端试图获取位于包含 m 个元素序列中索引为 `pos` 的数据时，只需构造一个“选择向量”：在该向量中除了 `pos` 处的数据设为 1 外，其余位置均设定为 0。

接着，客户端用公钥将这个选择向量依次加密并发送给服务器。服务器由于不具备私钥，接收到的仅仅是一组随机密文。此时，服务器遵循同态运算规则，将本地数据库的每一个元素分别与对应的选择密文进行标量乘法，并将所有乘积结果进行同态累加。

在这个求和过程中，仅有与密文“1”相乘的目标数据被保留了下来，而数据库中所有其他无关内容在与密文“0”相乘后均归零。最后，服务器将这个累加计算后的最终密文回传。客户端收到后使用私钥解密，就能在不泄露查询意图的前提下安全获取到目标数据。

3.4 扩展实验中的对称加密结构与数据类型转换

真实的隐私数据通常不仅限于简单的数值类型。如果将敏感文本直接存储在服务器，仍会存在数据泄露的风险。因此我们在扩展实验中，增设了基于对称加密（本实验依托基于 AES 的 Fernet 算法）的第二重防护：

- **对称加密保护**：在客户端本地留存对称密钥 k，预先对存入服务器的数据进行加密。以此确保即便服务器遭到破坏，未经授权者也无法解读数据的真实内容。
- **数据类型的转换难点**：对文本等内容进行对称加密后，生成的是计算机二进制字节流（Bytes 数据）。然而，Paillier 的同态数学运算要求所有参与计算的元素必须为大整数（Integer）。
- **实现逻辑**：为了解决该冲突，客户端需要对数据类型进行序列化转换。在存储前，将对称加密得到的字节流转换成等效的大整数形式并存入服务器。在查询时，这个大整数作为明文常量进入同态计算过程；当客户端获取并用 Paillier 私钥解密出这个大整数目标后，再执行反向操作：将其转回字节流，最后使用对称密钥 k 进行解密以恢复原始的字符串数据。这种嵌套的转换机制，极大地扩展了同态加密技术处理**不同类型数据**的能力。

4 实验环境配置流程

本次实验基于 Python 语言实现，主要依赖于 `phe`（Partially Homomorphic Encryption）库来实现 Paillier 同态加密算法，以及 `cryptography` 库来实现扩展实验中的对称加密机制。

4.1 安装与配置 phe 库

`phe` 库是 Python 中实现 Paillier 部分同态加密算法的开源规范库，封装了密钥生成、加密、解密及同态运算等底层细节。我们在实验环境终端中，执行以下命令完成安装：

```
pip install phe
```

在本次实验中，`phe.paillier` 模块所用到的核心函数如下：

- `paillier.generate_paillier_keypair()`：用于在客户端本地随机生成一对 Paillier 密钥。返回值为 `(public_key, private_key)` 元组。其中 `public_key` 将发送给服务器用于数据加密与同态运算，`private_key` 由客户端保管，用于最终计算结果的解密。

- `public_key.encrypt(value)`：公钥加密函数。将传入的明文常数（本次实验中为选择向量中的 0 或 1）转化为 Paillier 密文对象。该方法在底层会自动引入随机数机制，以保证加密结果的语义安全性。
- `private_key.decrypt(ciphertext)`：私钥解密函数。用于将服务器回传的累加同态密文对象逆向还原，提取出明文大整数。

4.2 安装与配置 cryptography 库

为了完成扩展实验，即在同态加密的基础上引入对称加密体系以实现双层数据保护，需要借助于专业的密码学工具包 `cryptography`。我们在实验环境终端中，执行以下命令完成安装：

```
pip install cryptography
```

在扩展实验的要求中，服务器端应当存储由客户端对称密钥加密后的密文，而不是直接存储原始状态的数据，以防范数据在服务端的明文留存风险。`cryptography.fernet` 模块提供了一套高度封装、符合 AES-128（CBC 模式）以及 HMAC 验证的安全对称加密方案（`Fernet`）。

在该库的支持下,客户端能够通过 `Fernet.generate_key()` 构建安全的对称密钥,利用 `.encrypt()` 对隐私字符串进行字节级加密掩盖，随后再依靠代码逻辑将字节流转化为大整数（Integer）形式以满足 Paillier 同态运算的数据类型规范。该库保证了扩展实验中保障数据在不被查询时处于安全加密状态。

5 基础实验

5.1 实现流程与思路

基础实验的核心目标是验证 Paillier 同态加密算法在实现隐私信息获取（PIR）时的可行性。具体实现流程分为以下几个阶段：

1. **数据准备阶段**：服务器端初始化一个包含若干整数元素的列表 `message_list`。在基础实验中，服务器中的数据是明文整数类型，以满足 Paillier 原生算法对大整数运算的兼容性要求。
2. **密文构造阶段**：客户端首先利用 `paillier.generate_paillier_keypair()` 生成专属的公私钥对。随后，客户端确定想要获取的数据索引位置 `pos`，并依此构造一个与服务器数据列表等长的选择向量。该向量在 `pos` 处的值设为 1，其余位置全为 0。为了隐藏真实意图，客户端使用公钥对选择向量中的每一个数字进行加密，生成包含大量随机数的密文向量，并将其发送给服务器。
3. **同态盲算阶段**：服务器接收到密文向量后，由于没有私钥，无法解析密文的具体内容。服务器直接在密文空间中执行运算：遍历自身的数据列表，将每一个明文数据与客户端发来的对应位置上的密文相乘，然后将所有乘积结果进行同态累加。根据 Paillier 的同态特性，与密文“0”相乘的数据将等效为 0，而与密文“1”相乘的数据将被完整保留。
4. **解密验证阶段**：服务器将最终累加计算得出的大密文传回给客户端。客户端收到后，使用只有自己持有的私钥进行解密，直接得到所需索引位置上的真实数值，从而完成隐私提取过程。整个流程中，客户端获取了数据，而服务器无从知晓该数据的具体位置。

5.2 具体实现代码

以下为基础实验的核心代码实现（`basic_pir.py`）：

```python
from phe import paillier
import random

def basic_pir_experiment():
    print("=== 基础实验：基于Paillier算法实现隐私信息获取 ===")

    # ----------------------------------------------------------
    # 1. 准备阶段
    # ----------------------------------------------------------
    # 服务器端保存的数值
    message_list = [100, 200, 300, 400, 500, 600, 700, 800, 900, 1000]
    length = len(message_list)
    print(f"[服务器] 拥有的数据列表：{message_list}\n")

    # ----------------------------------------------------------
    # 2. 客户端阶段（1）：生成公私钥，选择要读取的数据位置
    # ----------------------------------------------------------
    public_key, private_key = paillier.generate_paillier_keypair()

    # 随机选择一个想要获取的数据位置 (0 到 length-1)
    pos = random.randint(0, length - 1)
    print(f"[客户端] 想要获取的数据索引位置为：{pos}（该位置的实际数值应为
        {message_list[pos]}）")

    # 生成选择向量和密文向量
    select_list = []
    enc_list = []
    for i in range(length):
        # 如果是我们要找的位置，值为1，否则为0
        val = 1 if i == pos else 0
        select_list.append(val)
        # 将0或1进行加密，生成密文
        enc_list.append(public_key.encrypt(val))

    print(f"[客户端] 生成的选择向量（明文）：{select_list}")
    print("[客户端] 已将包含多个巨大随机数的密文向量(enc_list)发送给服务器...\n")

    # ----------------------------------------------------------
    # 3. 服务器阶段：利用同态特性进行盲运算
    # ----------------------------------------------------------
    print("[服务器] 收到客户端发送的密文请求，开始进行盲运算...")
    c = 0  # 初始累加值

    for i in range(length):
        # 核心逻辑：数据值 * 密文
        # 如果密文里藏的是 0，那么 x * E(0) = E(0)
```

```python
        # 如果密文里藏的是 1，那么 x * E(1) = E(x)
        c = c + message_list[i] * enc_list[i]

    print(f"[服务器] 产生最终密文：{c.ciphertext()}")
    print("[服务器] 已将计算后的这串密文返回给客户端...\n")

    # ----------------------------------------------------------
    # 4. 客户端阶段（2）：解密结果
    # ----------------------------------------------------------
    print("[客户端] 收到服务器返回的密文，使用私钥进行解密...")
    m = private_key.decrypt(c)
    print(f"[客户端] 最终解密得到的数值：{m}")

    if m == message_list[pos]:
        print("\n=> 【实验成功！】客户端成功获取了指定位置的数据，且在这个过程中，
        由于发送的都是密文，服务器根本不知道客户端获取的是哪一个数据。")
    else:
        print("\n=> 【实验失败！】")

if __name__ == "__main__":
    basic_pir_experiment()
```

5.3 代码运行结果与分析

以下为基础实验代码在终端中的执行输出结果：

图 5.1: 终端运行结果

```
=== 基础实验：基于Paillier算法实现隐私信息获取 ===
[服务器] 拥有的数据列表：[100, 200, 300, 400, 500, 600, 700, 800, 900, 1000]

[客户端] 想要获取的数据索引位置为：5（该位置的实际数值应为 600）
[客户端] 生成的选择向量（明文）：[0, 0, 0, 0, 0, 1, 0, 0, 0, 0]
```

```
[客户端] 已将包含多个巨大随机数的密文向量(enc_list)发送给服务器...

[服务器] 收到客户端发送的密文请求，开始进行盲运算...
[服务器] 产生最终密文: 27827323032632401194436004645602066111205710 2... (省略部
分密文数据) ...00041411103352338
[服务器] 已将计算后的这串密文返回给客户端...

[客户端] 收到服务器返回的密文，使用私钥进行解密...
[客户端] 最终解密得到的数值: 600

=> 【实验成功！】客户端成功获取了指定位置的数据，且在这个过程中，由于发送的都是密文，
服务器根本不知道客户端获取的是哪一个数据。
```

运行结果分析：

1. **符合实验约束**：根据要求，“从服务器给定的 m 个消息中获取其中一个，不得向服务器泄露获取了哪一个消息”。从输出可以看出，客户端最终成功提取了索引为 5，对应目标值为 600 的记录。在这个通信周期里，发送给服务器的查询请求全部为 Paillier 加密后的同态乱码元素序列，服务器端不执行任何公钥解密流程。因此，客户端请求对服务器表现出强隐私保护（服务器无法区分到底哪一个密文代表的是“1”）。

2. **客户端数据获取完成**：客户端使用私钥将由服务器计算的同态加密总和进行解密，最终只得到了自己确切想要的数据“600”，其他多余数据没有向用户本身造成冗余，这说明我们的基础的隐私信息获取机制成功完成。

6 扩展实验

6.1 实现流程与思路

在基础实验的框架之上，扩展实验旨在解决现实工程场景中更严苛的需求：在保证“查询意图隐私（PIR）”的同时，兼顾“数据存储隐私（对称加密防护）”。由于数据经对称加密后的形式（Bytes）与 Paillier 支持的形式（Integer）存在天然的类型冲突，本部分的实现思路如下：

1. **双重加密的数据预处理**：首先采用 `cryptography.fernet` 模块生成一个只保留在客户端的安全对称密钥 k。随后，选取需要存入服务器的原始敏感数据字符串。利用该对称密钥，将明文字符串逐一进行 AES 加密获得密文字节流。为了能够进行后续的同态运算，这些密文字节数据（Bytes）会被转换为对应的大整数（Integer）。转换后的大整数将存储在服务器中，用于响应后续的查询请求。

2. **PIR 密文匹配检索**：客户端根据与基础实验相同的逻辑，构建由 0 和指定位置的 1 组成的查找向量，用 Paillier 公钥将其逐个加密并发送请求。服务端对存库数据执行标量乘与同态累加操作：此时服务器存储的是转换后的大整数密文，因此同态运算得到的结果，实际上也就是目标密文大整数的同态加密形式。

3. **解密与类型复原验证**：当包含计算结果的同态密文返回至客户端后，按以下步骤还原数据：

   - 首先利用客户端的 Paillier 私钥进行解密，提取出代表对称加密结果的中间形态大整数；

- 随后使用 Python 的反向转换函数（`.to_bytes()`），将其还原为 Fernet 算法生成的加密字节流；
- 最终使用客户端保存的对称密钥 k 对字节流进行解密并解码，还原出明文字符串数据。

6.2 具体实现代码

以下为扩展实验的核心代码实现（`extended_pir.py`）：

```python
from phe import paillier
import random
from cryptography.fernet import Fernet

def extended_pir_experiment():
    print("=== 扩展实验：结合对称加密的隐私信息获取 ===\n")

    # ============================================================
    # 步骤 1：客户端生成对称密钥，并准备初始数据
    # ============================================================
    print("--- [阶段 1：数据准备与对称加密] ---")
    # 客户端生成对称密钥 k（使用 Fernet）
    symmetric_key = Fernet.generate_key()
    cipher_suite = Fernet(symmetric_key)
    print(f"[客户端] 生成对称密钥 k: {symmetric_key}")

    # 假设我们有以下这些敏感信息的明文列表（字符串）
    plaintext_list = [
        "Confidential Data A",
        "Confidential Data B",
        "Confidential Data C",
        "Confidential Data D",
        "Confidential Data E"
    ]
    length = len(plaintext_list)
    print(f"[数据源] 原始明文数据：{plaintext_list}\n")

    # 客户端（或数据拥有者）将数据用对称密钥加密，并转换为大整数，然后存放到服务器
    server_database = []
    for pt in plaintext_list:
        # 1. 字符串转 utf-8 bytes
        pt_bytes = pt.encode('utf-8')
        # 2. 用对称密钥进行 AES 加密，得到密文 bytes
        enc_bytes = cipher_suite.encrypt(pt_bytes)
        # 3. 【核心技术点】：将密文 bytes 转换为大整数，因为 Paillier 只能处理数字！
        enc_int = int.from_bytes(enc_bytes, byteorder='big')
        server_database.append(enc_int)

    print("[服务器] 已接收并存储了用客户端密钥加密后的数据（以大整数形式保存）:")
    for i, data in enumerate(server_database):
        print(f"  索引 {i}: {str(data)[:20]}...（省略过长数字）")
```

```python
    print()


    # ============================================================
    # 步骤 2: 客户端发起隐私获取请求 (过程与基础实验几乎一样)
    # ============================================================
    print("--- [阶段 2: 客户端发起 PIR 请求] ---")
    public_key, private_key = paillier.generate_paillier_keypair()

    # 客户端想获取索引为 2 的数据 (即 "Confidential Data C")
    pos = 2
    print(f"[客户端] 想要获取的数据索引位置为: {pos}")

    select_list = []
    enc_list = []
    for i in range(length):
        val = 1 if i == pos else 0
        select_list.append(val)
        enc_list.append(public_key.encrypt(val))

    print("[客户端] 已将加密后的选择向量发送给服务器...\n")


    # ============================================================
    # 步骤 3: 服务器进行盲运算
    # ============================================================
    print("--- [阶段 3: 服务器盲运算] ---")
    print("[服务器] 正在基于同态加密进行安全查询...")
    c = 0
    for i in range(length):
        # 这里的 server_database[i] 虽然是密文, 但现在表现为一个大整数
        # 用它去乘以 Paillier 密文, 利用同态特性将其累加
        c = c + server_database[i] * enc_list[i]

    print("[服务器] 运算完成, 将最终的 Paillier 密文返回给客户端。\n")


    # ============================================================
    # 步骤 4: 客户端双重解密拿回明文
    # ============================================================
    print("--- [阶段 4: 客户端双重解密] ---")
    # 第一重解密: Paillier 解密
    retrieved_int = private_key.decrypt(c)
    print(f"[客户端] 1. 通过 Paillier 私钥解密, 得到大整数: {str(retrieved_int)[:20]}...")

    # 第二重解密: 对称解密
    # 1. 先把大整数转换回密文 bytes
    # 计算大整数需要多少个 byte 才能装下
```

```python
    byte_length = (retrieved_int.bit_length() + 7) // 8
    retrieved_bytes = retrieved_int.to_bytes(byte_length, byteorder='big')

    # 2. 使用对称密钥解密恢复明文
    try:
        decrypted_pt_bytes = cipher_suite.decrypt(retrieved_bytes)
        final_plaintext = decrypted_pt_bytes.decode('utf-8')
        print(f"[客户端] 2. 通过对称密钥 k 解密，成功还原明文：>>> {final_plaintext} <<<")

        if final_plaintext == plaintext_list[pos]:
            print("\n=> 【扩展实验成功！】成功在服务器端不清楚查询意图、且数据本身被对称加密保护的双重安全环境下，获取了目标数据！")
    except Exception as e:
        print(f"\n=> 【实验失败】对称解密出错：{e}")

if __name__ == "__main__":
    extended_pir_experiment()
```

6.3 代码运行结果与分析

以下为扩展实验代码在终端中的执行输出结果：

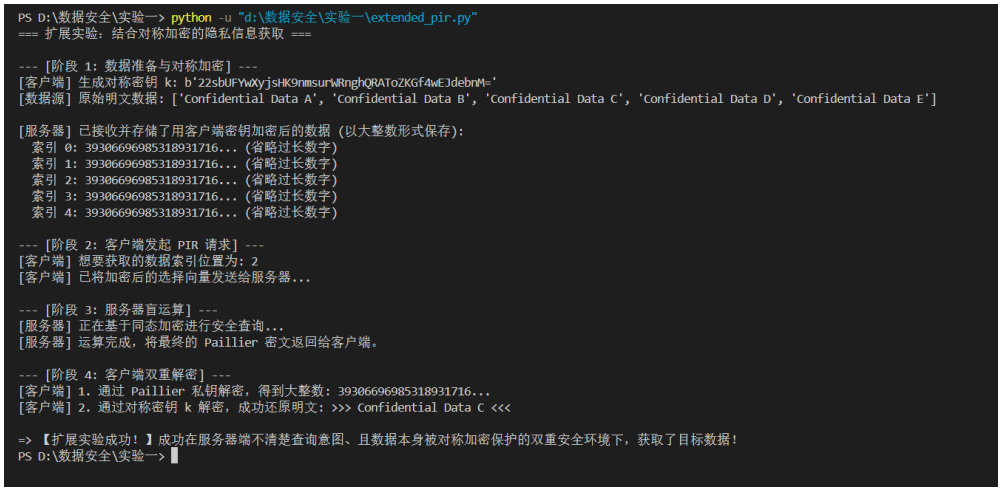

图 6.2: 终端运行结果

```
=== 扩展实验：结合对称加密的隐私信息获取 ===

--- [阶段 1: 数据准备与对称加密] ---
[客户端] 生成对称密钥 k: b'22sbUFYwXyjsHK9nmsurWRnghQRAToZKGf4wEJdebnM='
[数据源] 原始明文数据: ['Confidential Data A', 'Confidential Data B', 'Confidential Data C', 'Confidential Data D', 'Confidential Data E']

[服务器] 已接收并存储了用客户端密钥加密后的数据 (以大整数形式保存):
  索引 0: 39306696985318931716... (省略过长数字)
```

```
  索引 1: 39306696985318931716... (省略过长数字)
  索引 2: 39306696985318931716... (省略过长数字)
  索引 3: 39306696985318931716... (省略过长数字)
  索引 4: 39306696985318931716... (省略过长数字)

--- [阶段 2: 客户端发起 PIR 请求] ---
[客户端] 想要获取的数据索引位置为: 2
[客户端] 已将加密后的选择向量发送给服务器...

--- [阶段 3: 服务器盲运算] ---
[服务器] 正在基于同态加密进行安全查询...
[服务器] 运算完成, 将最终的 Paillier 密文返回给客户端。

--- [阶段 4: 客户端双重解密] ---
[客户端] 1. 通过 Paillier 私钥解密, 得到大整数: 39306696985318931716...
[客户端] 2. 通过对称密钥 k 解密, 成功还原明文: >>> Confidential Data C <<<

=> 【扩展实验成功!】成功在服务器端不清楚查询意图、且数据本身被对称加密保护的双重安全
环境下, 获取了目标数据!
```

运行结果分析:

1. **实现了服务器端数据的机密性保护**: 日志输出表明, 客户端生成密钥 k 后, 服务器接收和存储的不再是如"Confidential Data A" 形式的原始明文, 而是经过对称加密并转换得到的大整数。这符合扩展实验关于 "在服务器端存储用对称密钥加密密文" 的目标要求。

2. **支持更通用的数据类型提取**: 基础实验的运算仅基于整数, 而在引入 `Bytes <-> Integer` 的互相转换后, 系统具备了处理普遍字符串文本数据的能力。在查询阶段, 客户端获取了索引为 2 的检索结果, 并成功解密还原为最初的明文字符串。这说明数据类型转换机制能够有效配合 Paillier 算法来完成信息的传递。

3. **验证了目前的双重加密机制**: 运行结果验证了在当前架构下两道防护机制的有效性。服务器既获取不到解开数据的对称密钥 (保障存储数据隐私), 也无法知晓用户的选择向量究竟指向哪个位置 (保障查询动作隐私), 但系统依然能够正确且正常地向用户返回目标文件。这验证了本扩展方案逻辑的合理性与安全性。

7 实验总结

7.1 主要工作

在本次实验中, 我们围绕 "基于 Paillier 同态加密的隐私信息获取 (PIR)" 这一核心主题进行了实验, 完成了以下工作:

1. **原理分析与环境搭建**: 我们首先深入研究了 Paillier 加密体系的基本原理, 尤其是加法同态和标量乘法特性的数学基础。然后配置了基于 Python 的实验环境, 并安装了 `phe` 和 `cryptography` 底层结构库。

2. **基础 PIR 模型的实现构建**：根据基础实验要求，编写了 `basic_pir.py` 实现了基础的数据隐私查询机制。通过在客户端构建“全 0 单 1”的加密选择向量，借助服务器端在密文空间的同态盲运算累加机制，成功完成了向服务器索要具体数据过程中目标元素的隐藏。

3. **扩展实验中的双重加密机制**：为解决服务端数据防泄漏的存储隐私问题，在 `extended_pir.py` 中增设了基于 AES-128（Fernet）的对称加密。通过解决并实现密文 `Bytes` 字节流与 `Integer` 大整数的互相转换，成功串联了两套不同的加密算法，使数据安全性提高。

7.2 实验心得

通过本次基于 Paillier 同态加密的隐私信息获取（PIR）实验，我将课上所学的密码学理论成功转化为了具体的工程代码，收获颇丰。在基础实验中，我借助同态加密的“标量乘法”特性，直观地体会了如何利用加密的选择向量在服务器端进行盲运算累加，从而实现对“查询意图”的隐藏；在扩展实验环节，我学习了 AES 对称加密的实际运用，以及密文字节流（Bytes）与同态大整数（Integer）之间的数据类型转换。这次实验让我深刻认识到，在真实场景下，数据安全防护并不是单一算法的简单调用，而是需要将多套密码体系相互融合，从而构建出更加安全的保密机制，加强了我对于实际场景下数据安全的认识。

8 github 仓库

代码详见： https://github.com/Wangxuuuuu/Data-Security/tree/main